

ANGULAR API CALL

Create Component name: getAPIEX

get-apiex.ts

```
export class GetAPIEx {
  http = inject(HttpClient);
  userlist: WritableSignal<any[]> = signal([]);
  constructor() {
    this.getAllusers();
  }
  getAllusers() {
    this.http.get("https://jsonplaceholder.typicode.com/users").subscribe({
      next: (res: any) =>
      {
        debugger;
        this.userlist.set(res)
      },
      error: (erro: any) =>
      {
        alert("Unable to load users");
      }
    })
  }
}
```

Get-apiex.html

```
<ul>
  @for(item of userlist(); track $index)
  {
    <li>{{item.name}} {{item.email}} </li>
  }
</ul>
```

Code Explanation:

http = inject(HttpClient);

This creates an object of Angular's **HttpClient** using the new Angular dependency injection syntax.

Internally,

Angular already has one HttpClient object.

Angular Injector



HttpClient Object



inject(HttpClient)

Now,

'http' points to that object.

So later you can write

this.http.get(...)

Constructor

```
constructor() {  
  this.getAllusers();  
}
```

Constructor

Runs automatically when the component object is created.

Execution

Angular creates component



Constructor executes



getAllusers() called

Suppose page loads.

Execution becomes

Page opens



Component created



Constructor runs



getAllusers()



API call

Why subscribe()?

.subscribe({
HTTP calls take time.

Maybe

- 1 second
- 2 seconds
- 500 milliseconds

Angular **doesn't wait**.

Instead

Request sent

↓

Continue executing other code

↓

When response arrives

↓

subscribe() executes

This is called Asynchronous Programming